sqli

BDX I/O

Specification Driven Development
Développer à l'ère des LLMs

Philippe Bousquet – Architecte & Leader Java Community SQLI
Camille Marti – Chef de projet

Qui sommes-nous ?

BDX I/O



Philippe Bousquet

- Architecte & Leader Java Community SQLI
- Passionné de dev depuis mes 9 ans
- Près de 30 ans d'expérience professionnelle
- Pilote la Veille & l'Innovation Java
- Ambassadeur IA sur Bordeaux



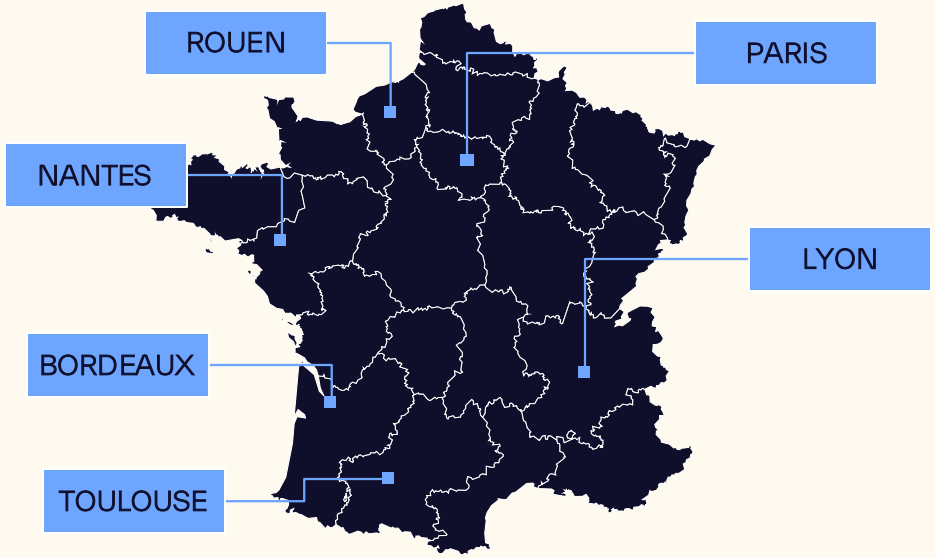
Camille Marti

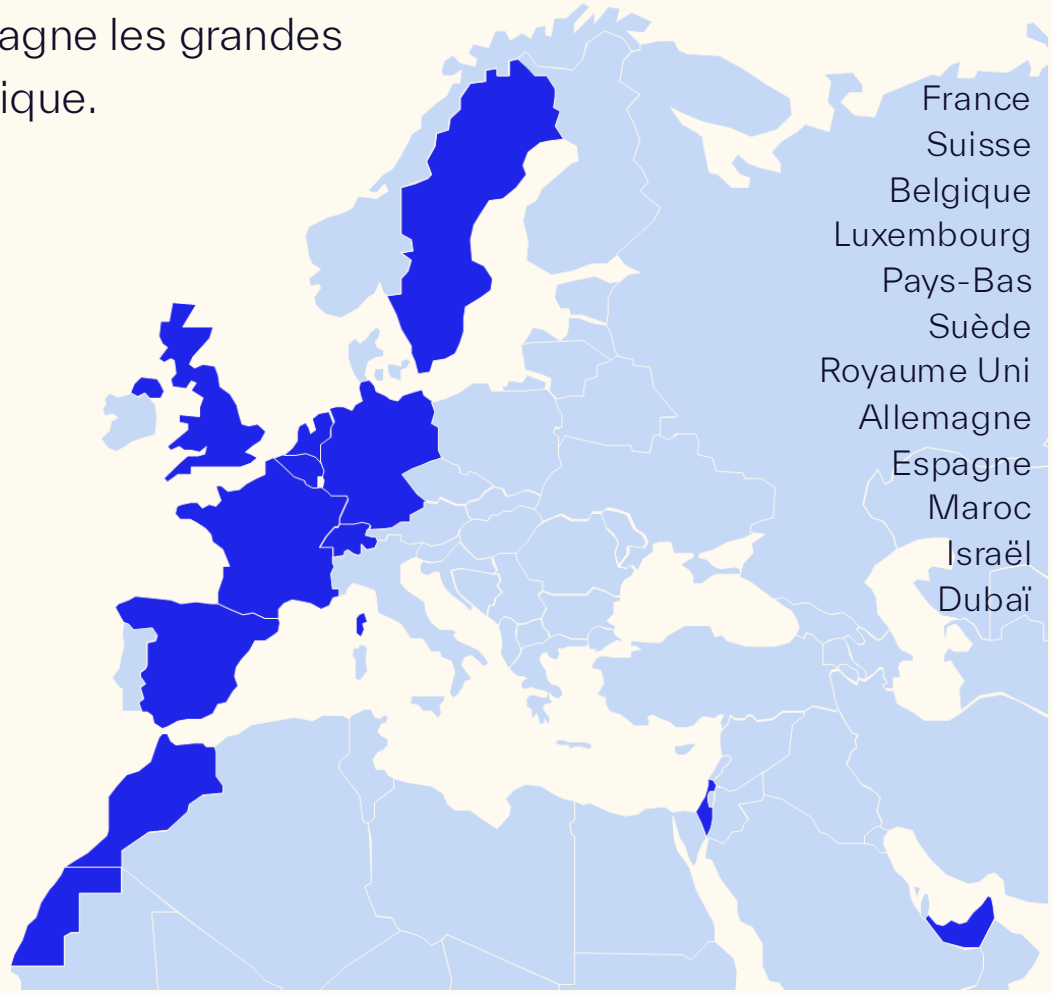
- Chef de projet
- Anime des équipes depuis 15 ans maintenant
- Passé par des structures comme une start-up, TPE, PME et ESN
- Soutien psychologique de Philippe en cas de besoin

Le Groupe SQLI

BDX I/O

SQLI est un groupe européen de services numériques qui accompagne les grandes marques internationales dans la création de valeur grâce au numérique.





1990
Création du groupe

253
M€ en 2025

2200
Employés

12
Pays

BDX I/O

Specification Driven Development
Un peu de théorie

BDX I/O

Specification Driven Development
Mais d'abord un peu d'histoire !

Une évolution des façons de travailler...

BDX I/O

Au début du XXème siècle : Organiser et planifier

En 1950 : Le bon vieux cycle en V

Années 1980-1990 : Renforcement du management de projet

A partir des années 1990 : Emergence de l'agilité

Fin 90 : Le Test-Driven Development (ou TDD)

Année 2000 : Le Behavior-Driven Development (ou BDD)

Et puis ces dernières années, l'IA a fait un énorme boom... Et on arrive au Spec-Driven Development

DÉVELOPPEMENT LOGICIEL

Une évolution fascinante

Une même ambition : livrer plus de valeur, plus sereinement.

Des produits
qui créent de la valeur,
durablement.

01 Cycle en V (Waterfall)

"Écris tout ce que nous voulons construire avant de commencer."

Tout planifier
en amont.



PRÉVOIR
Un besoin maîtrisé

02 Agile

"Nous ne pouvons pas tout savoir au départ, avançons par petites itérations."

Apprendre
et s'adapter.



ITÉRER
Face à l'incertitude

03 Test-Driven Development (TDD)

"Écrivons d'abord le test qui décrit ce que le code doit faire."

Des tests pour
guider le code.



SÉCURISER
La qualité du code

04 Behavior-Driven Development (BDD)

"Décrivons d'abord le comportement attendu par l'utilisateur / métier."

Un langage commun
autour du comportement.



ALIGNER
Métier, tech, utilisateurs

05 Spec-Driven Development

"Formalisons suffisamment l'intention et les exigences pour guider l'implémentation et sa validation."

Une spécification
au service de l'implémentation
(et de l'IA).



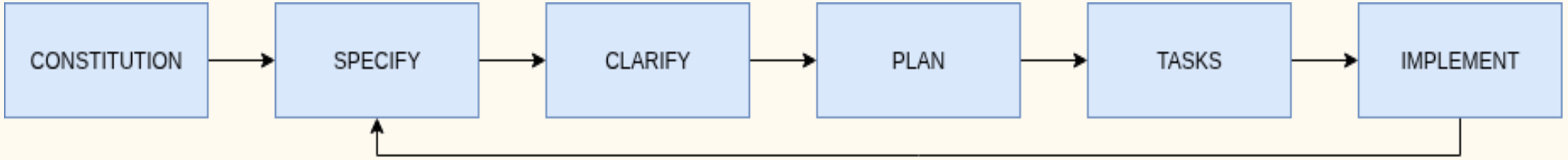
CONSTRUIRE DEMAIN
Des produits plus utiles

DIFFÉRENTES APPROCHES, UNE MÊME LOGIQUE DE **PROGRÈS**.

Et l'histoire continue...

sqli

Spec-Kit
Un framework qui discipline l'IA



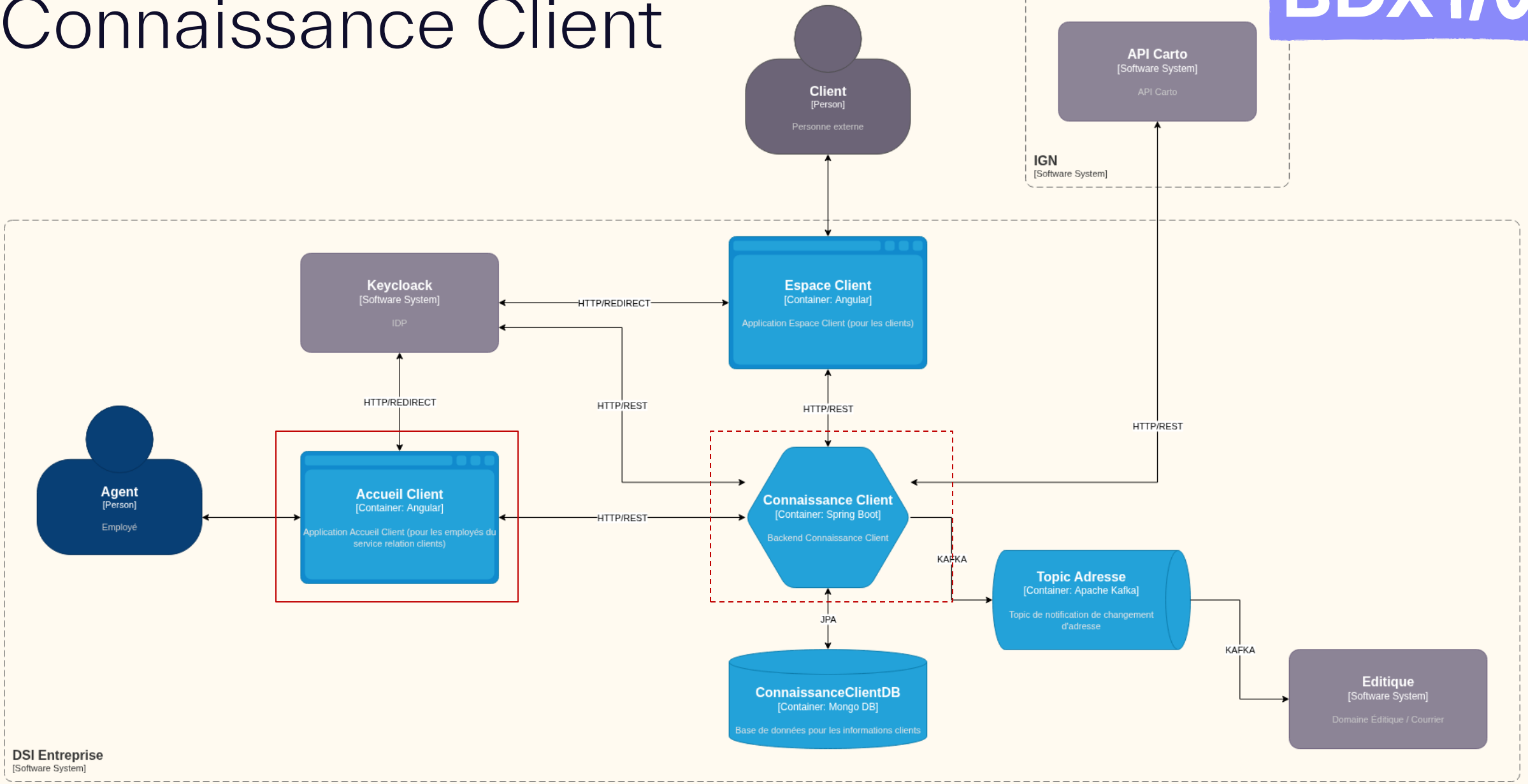
- Constitution : Principes d'architecture & développement
- Spécification & Clarification : Spécification fonctionnelle sans ambiguïté
- Plan : Plan d'implémentation technique
- Tâches : Liste de tâches unitaires
- Implémentation : Génération itérative

BDX I/O

Notre champ d'expérimentation
Connaissance Client

Connaissance Client

BDX I/O



Les prérequis

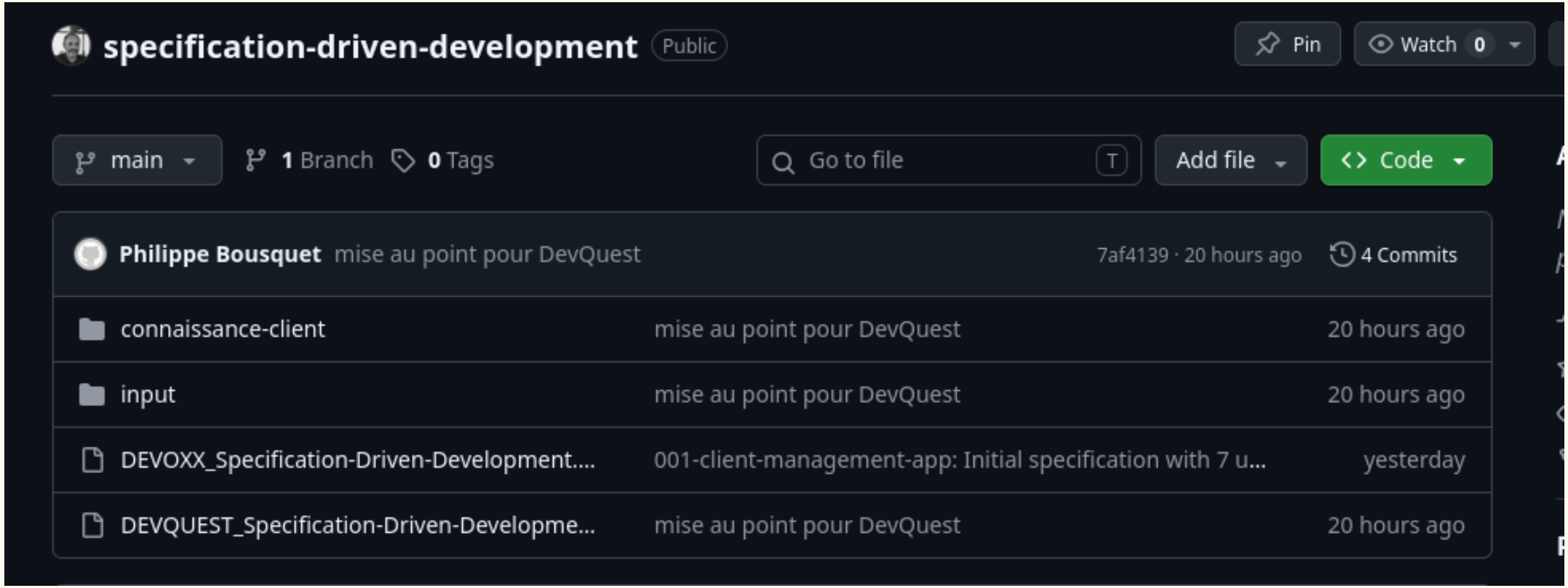
BDX I/O

Ce dont vous aurez besoin

- Git
- Un IDE avec Github Copilot (ou un autre assistant IA)
- UV (https://docs.astral.sh/uv/getting-started/installation/)
- React ou Angular, NPM
- Java 21, Maven (si on a du temps)
- Des connaissances en développement (préférable)

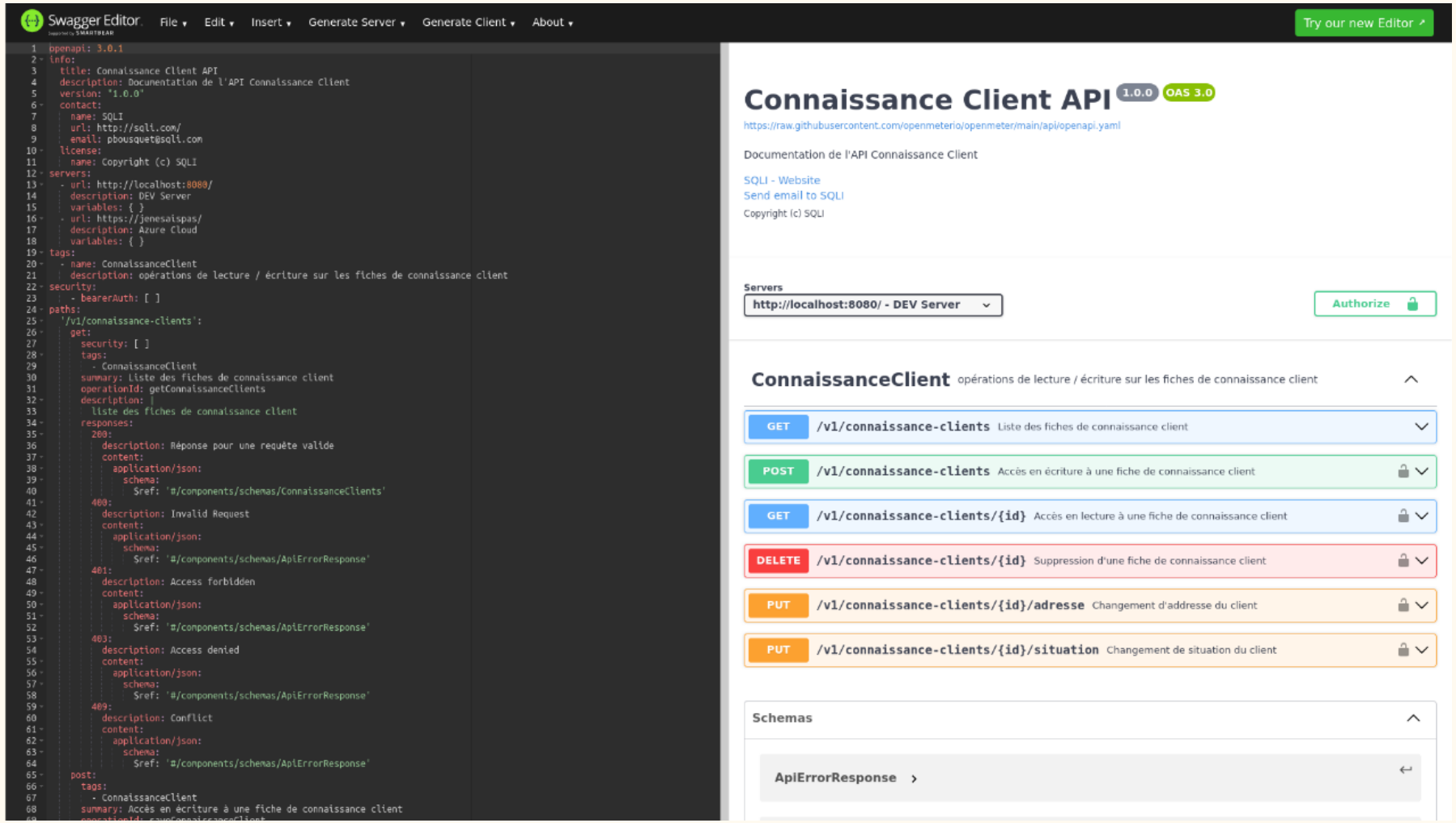
Récupérer le Repository

https://github.com/darken33/specification-driven-development



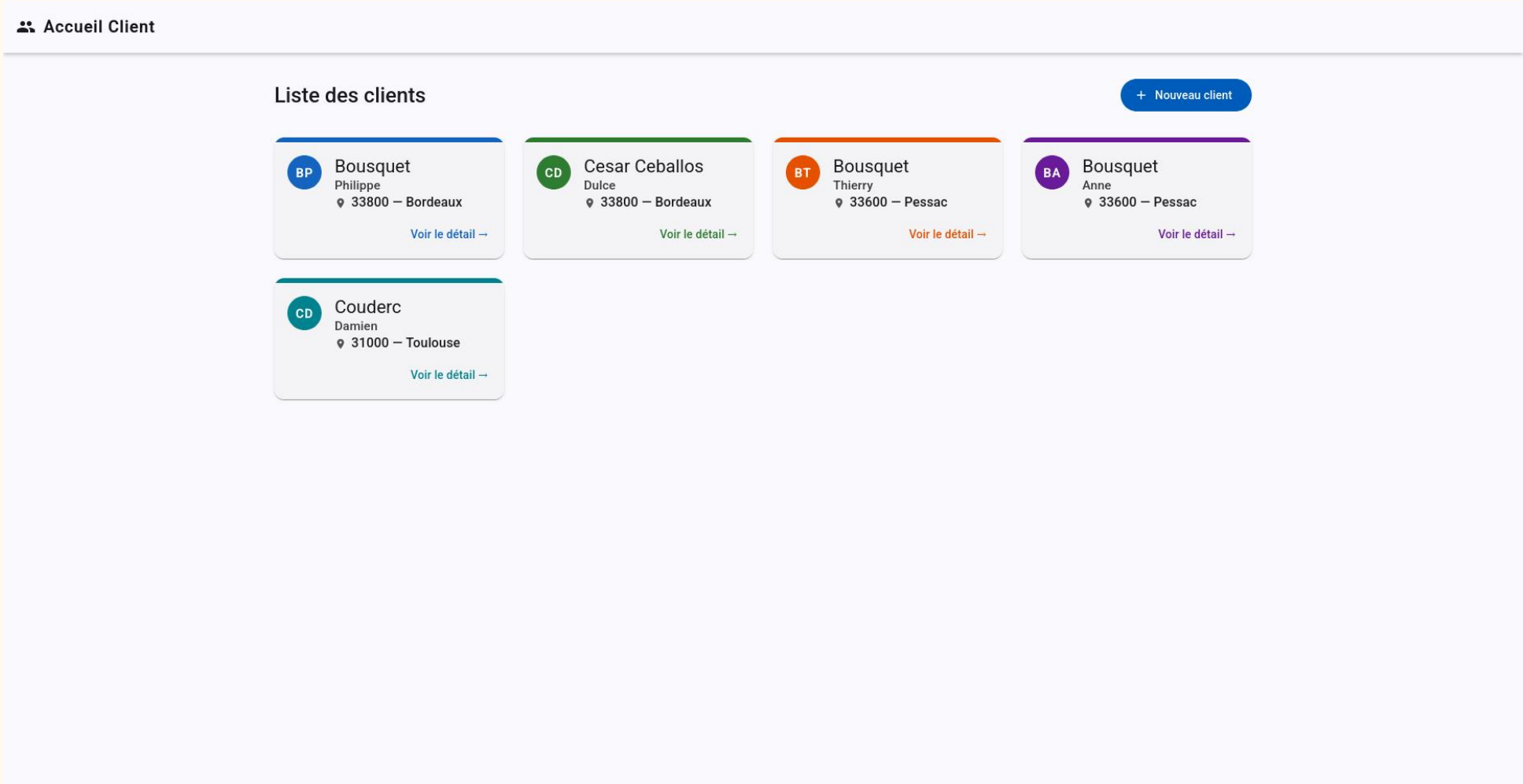
Un contrat OpenApi

connaissance-client-api.yaml



Des maquettes

maquette_accueil-client.png



BDX I/O

Accueil Client

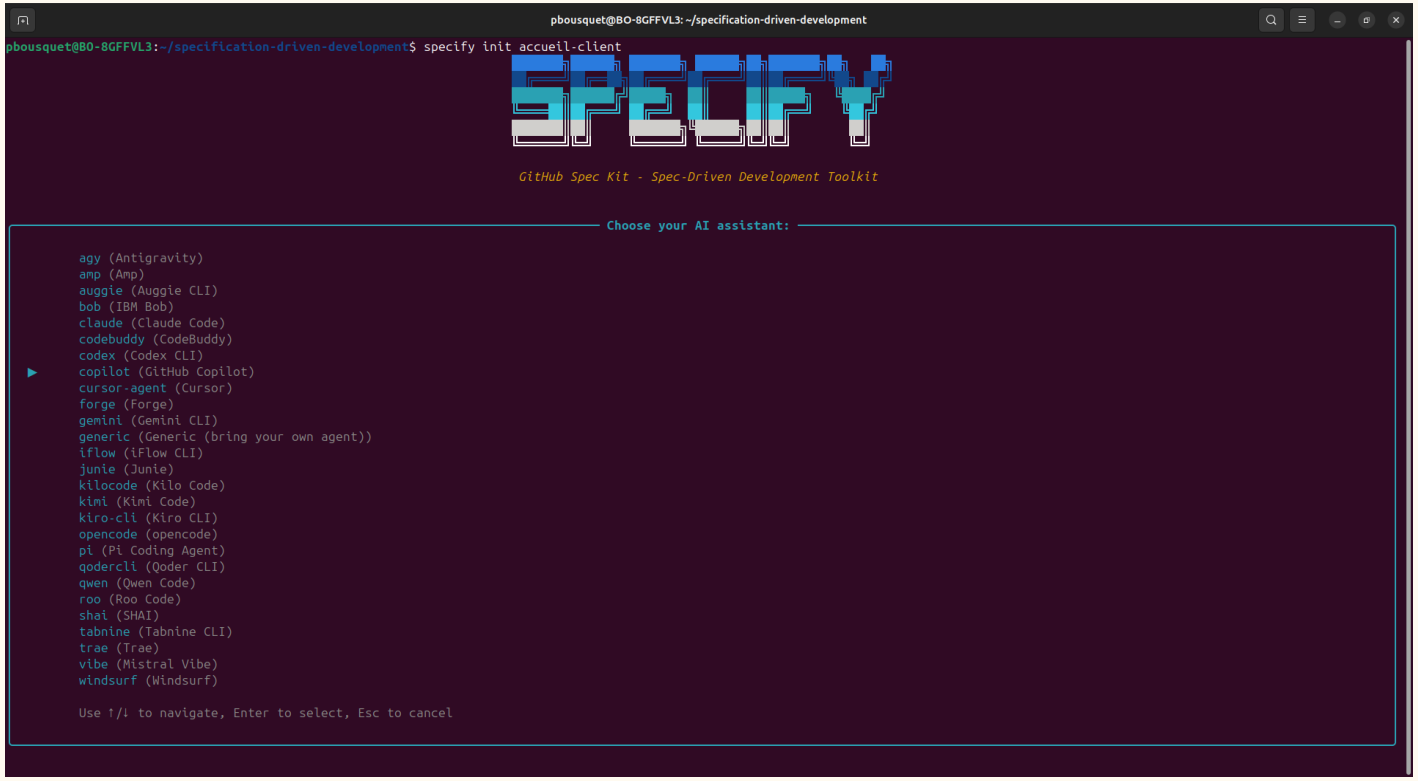
Créer une nouvelle application (Frontend)

Initialisation de Spec-Kit

Via la CLI

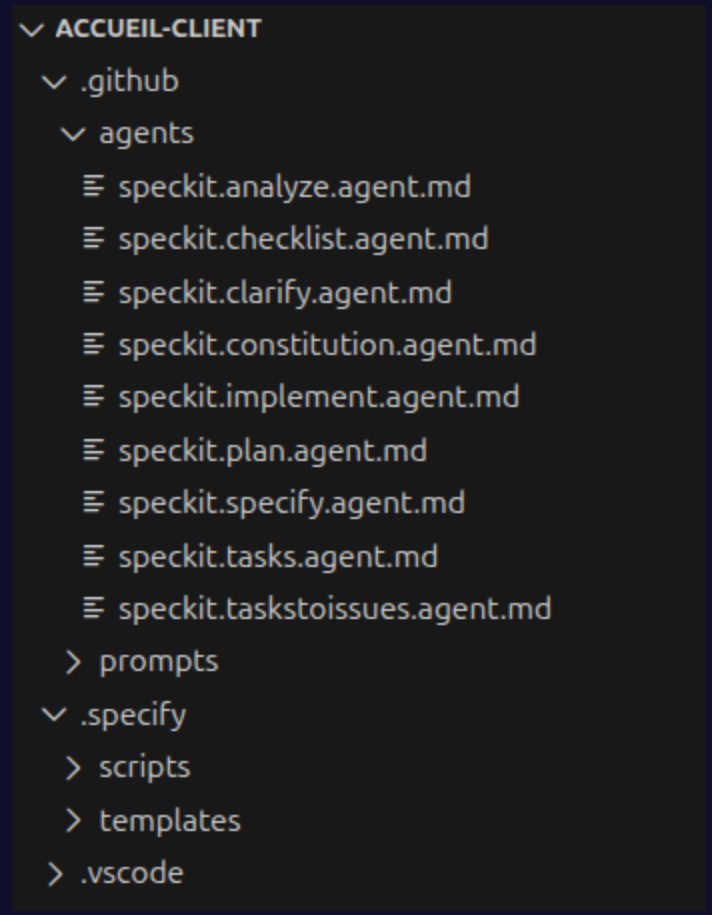
```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

```
specify init accueil-client
```



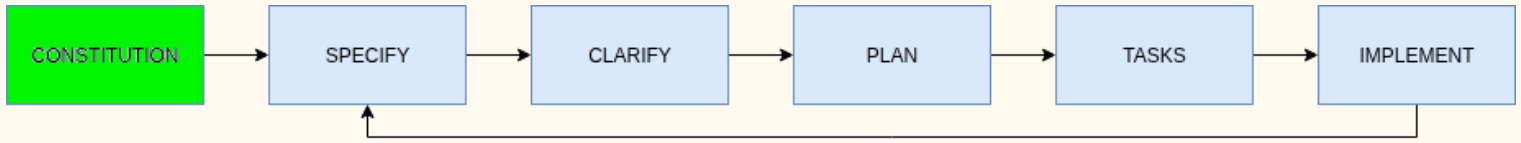
BDX I/O





💻 Demo / VS Code

Constitution



Exemple de prompt :

```
/speckit.constitution En tant qu'architecte technique spécialisé en Angular et
TypeScript, définis la structure, les principes architecturaux et de
développement d'une application front-end moderne. Pour ce faire, suis les
meilleures pratiques actuelles, en respectant au minimum les principes
suivants :

* Utiliser les dernières versions d'Angular et de TypeScript.
* Utiliser en priorité les outils d'industrialisation et génération ng et npm
* Principe KISS : Privilégier la simplicité.
* Séparer explicitement l'interface utilisateur du traitement.

S'agissant d'un POC, n'ayant pas pour but de partir en production, on ne
traitera pas les éléments suivants :

* Authentification et Sécutrité
* Pipelines de CI/CD

Définis uniquement le fichier constitution.md, puis me le retourner afin que
je puisse vérifier le respect de ces principes.
```

BDX I/O

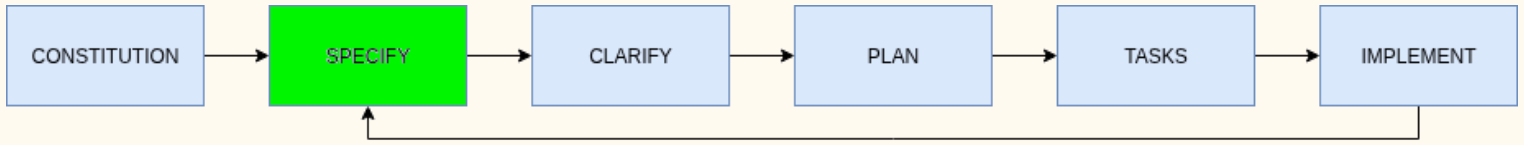


Principes attendus :

- Angular & TypeScript à jour
- KISS (Keep It Simple, Stupid)
- Séparation UI vs Traitement
- Mode POC

💻 Demo / VS Code

Spécifications



Exemple de prompt :

```
/speckit.specify Nous disposons actuellement d'une application backend exposant des
API REST définies dans le contrat OpenAPI (fichier `connaissance-client-api.yaml`).
En tant qu'architecte technique et fonctionnel possédant de solides compétences en
UX/UI, Tu dois spécifier une application frontend de gestion de clients qui accède à
tous les points de terminaison du contrat OpenAPI.

Nous disposons des maquettes suivantes :
* Page d'accueil / liste des clients `maquette_accueil-client.png`.
* Page nouveau client `maquette_nouveau-client.png`.
* Page détail client `maquette_detail-client.png`.
* Pop Up modification d'adresse `maquette_modifier-adresse.png`.
* Pop Up modification situation `maquette_modifier-situation.png`.
* Pop Up supprimer client `maquette_supprimer-client.png`.

Nous allons développer une première version MVP consistant à disposer de la page
d'accueil, le backend API n'étant pas disponible il faudra disposer d'un mock pour
les tests.

Une fois la spécification réalisée, rends moi la main afin que je valide ce que tu
auras rédigé.
```

BDX I/O

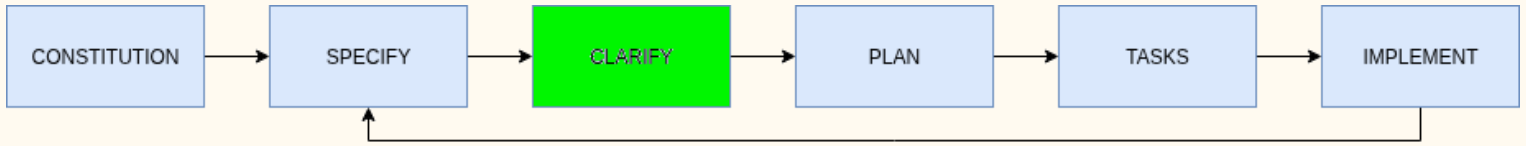


Spécification attendue :

- Une US / Maquette – endpoint API
- Un Mock service API
- Une Architecture
- Un MVP
- Diagramme de séquence ?

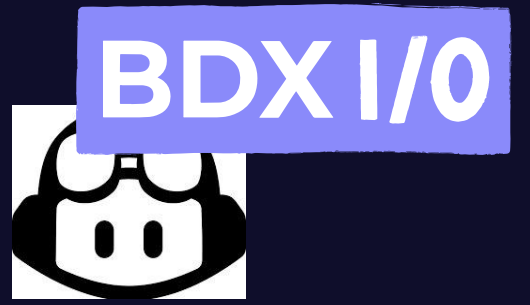
💻 Demo / VS Code

Clarification



Exemple de prompt :

```
/speckit.clarify Clarifies la spécification ci-jointe si nécessaire.
```

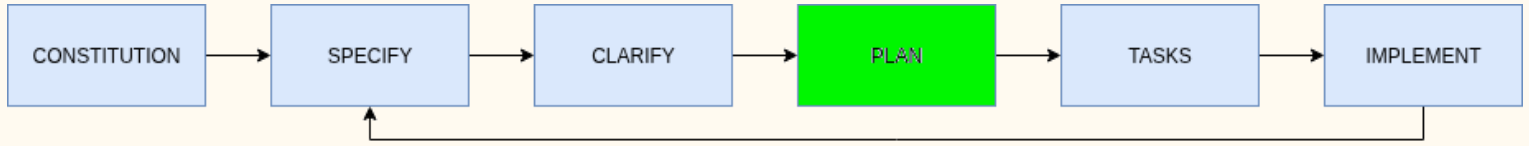


Passage en mode interactif

- Liste vide ?
- Mock Dataset ?
- Spinner ?
- Contrôles de surface ?
- Couleurs des cartes ?

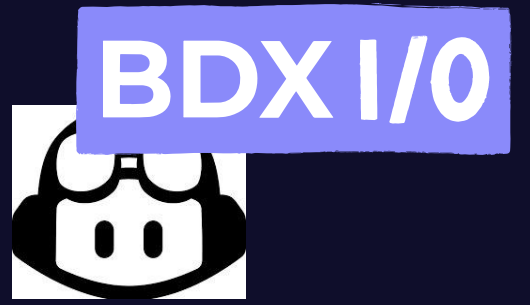
💻 Demo / VS Code

Définir le plan



Exemple de prompt :

```
/speckit.plan Crées un plan pour la spécification ci-jointe. Une fois le plan établi rends moi la main afin que je le valide.
```

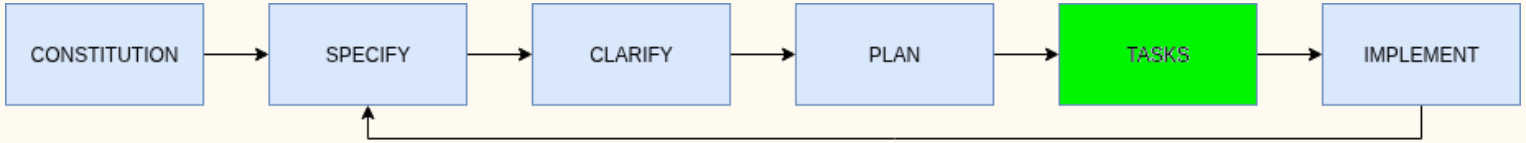


Plan défini

- Phase 1: Angular project setup
- Phase 2: Foundation (mock API + models)
- Phases 3-7: User stories
- Phase 8: Polish & testing

💻 Demo / VS Code

Transformer le plan en tâches



Exemple de prompt :

```
/speckit.tasks Transforme le plan ci-joint en tâches. Une fois les
tâches établies rends moi la main afin que je les valide.
```

BDX I/O

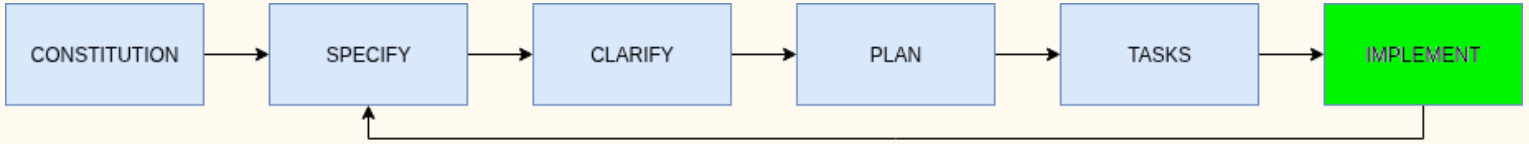


Tâches définies

- 8 phases
- N tâches

💻 Demo / VS Code

Démarrer l'implémentation



Exemple de prompts :

```
/speckit.implement Implémente la phase 1, puis rend moi la main afin que je puisse valider, l'idéal serait de pouvoir lancer l'application pour voir le rendu initial.
```

```
/speckit.implement Lance l'implémentation par phase à partir de la liste des tasks, en suivant le plan et les spec.
```

Améliorer l'interface utilisateur :

```
/speckit.implement Parfait cela fonctionne, par contre le design n'est pas excellent, peux-tu me proposer un bandeau design pour le titre, et plus de couleur sur les cartes clients ?.
```



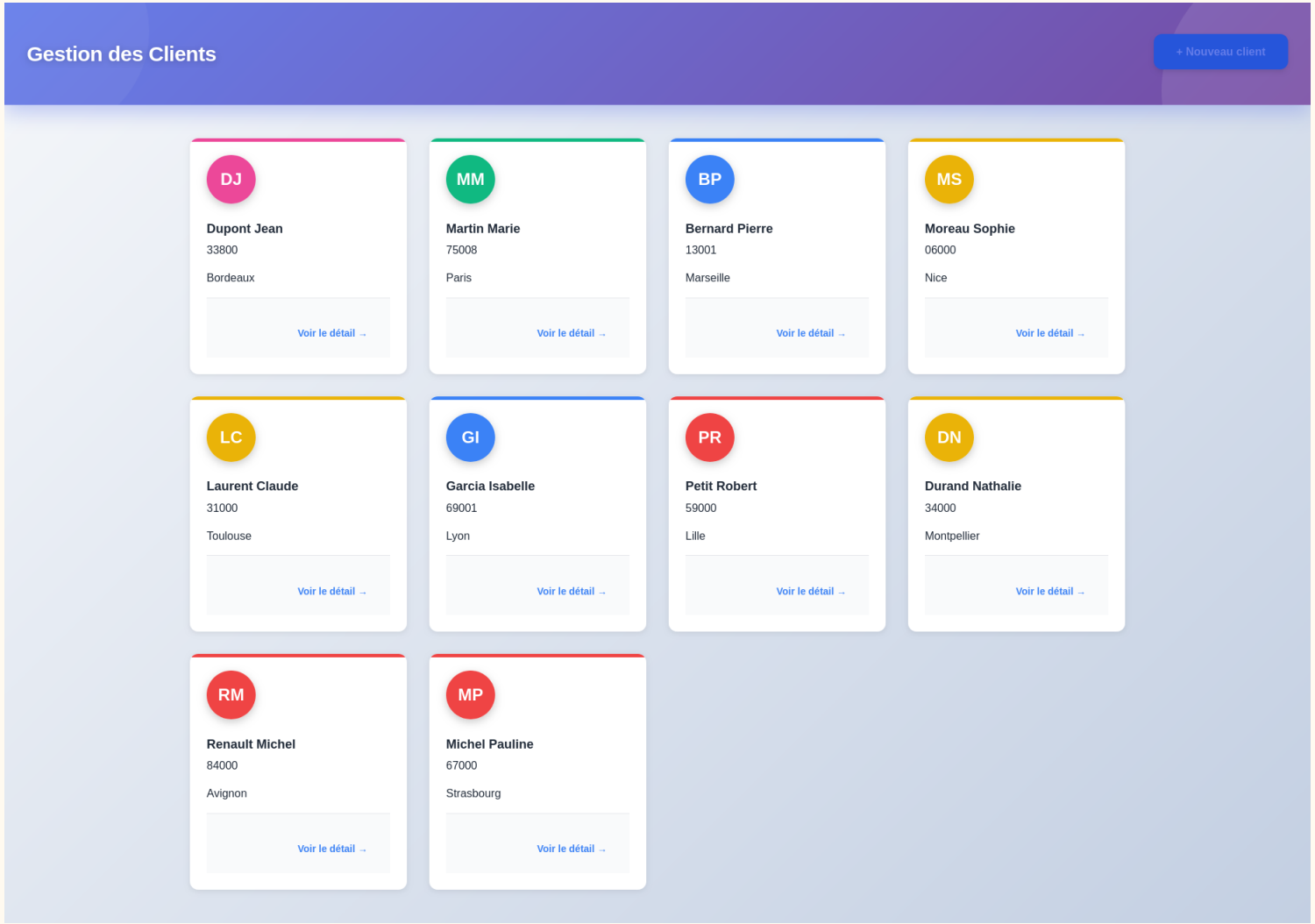
Passage en cycle itératif

- Phase 1 -> Setup
- Phase 2 -> Fondations
- Phase 3 -> User Story 1
- ...

Correctifs si nécessaire

💻 Demo / VS Code

Notre application générée



BDX I/O

Connaissance Client

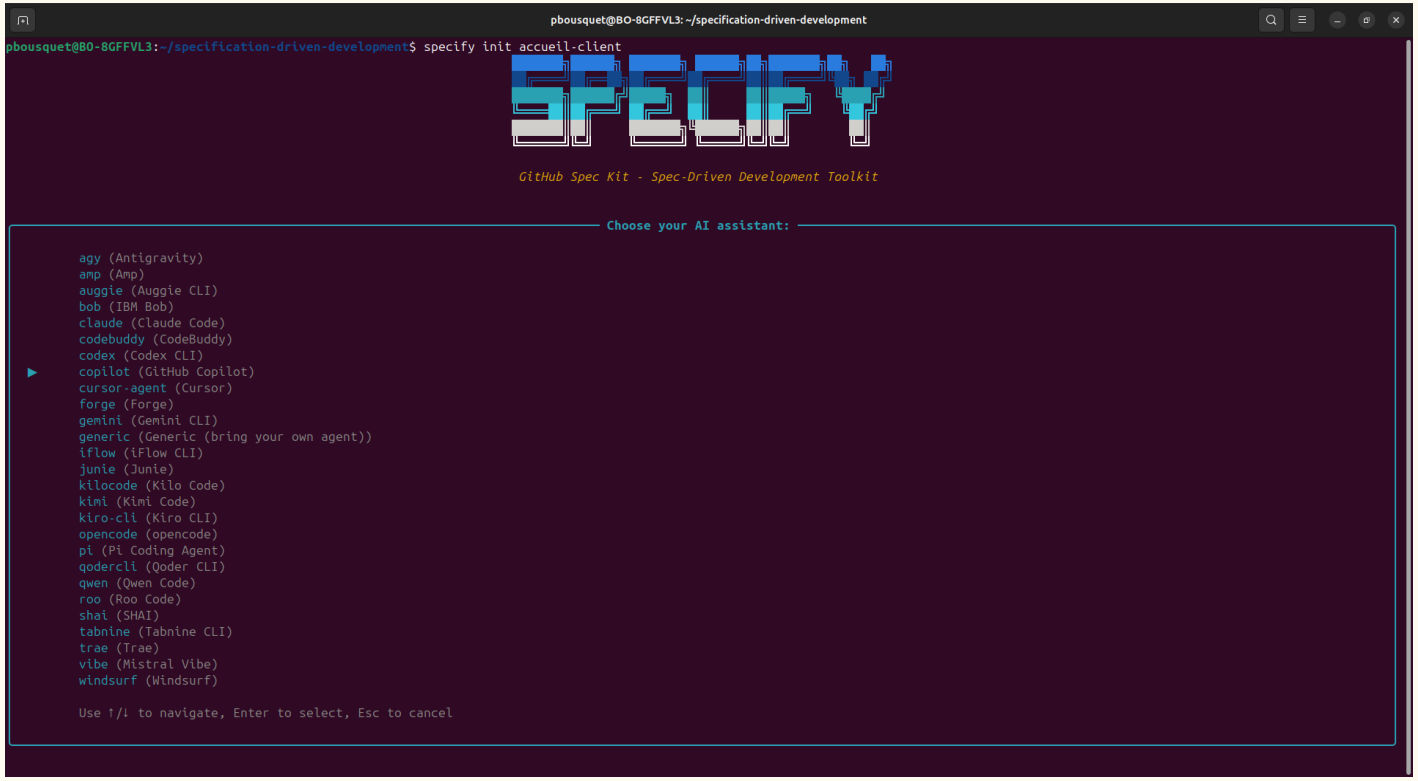
Reprise d'une application existante (Backend API)

Initialisation de Spec-Kit

Via la CLI

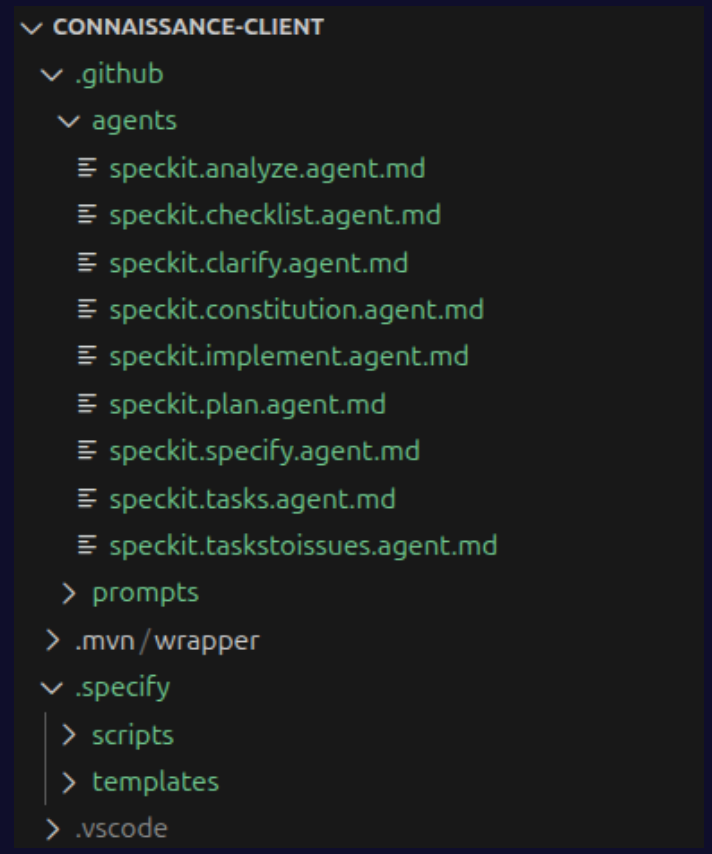
```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

```
cd connaissance-client && specify init --here
```



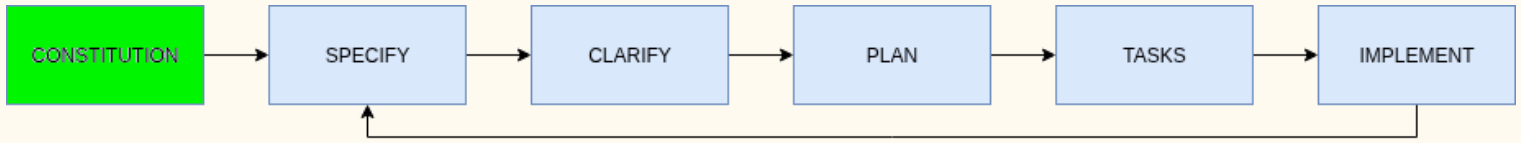
BDX I/O





💻 Demo / VS Code

Constitution



Exemple de prompt :

```
/speckit.constitution En tant qu'architecte technique spécialisé en Java et Spring-Boot, effectue une analyse complète du projet afin d'en déterminer les principes d'architecture et les pratiques de développements afin d'alimenter le constitution.md. Une fois la constitution rédigée, rends-moi la main afin que je procède à sa validation avant de passer à l'étape suivante.
```

BDX I/O

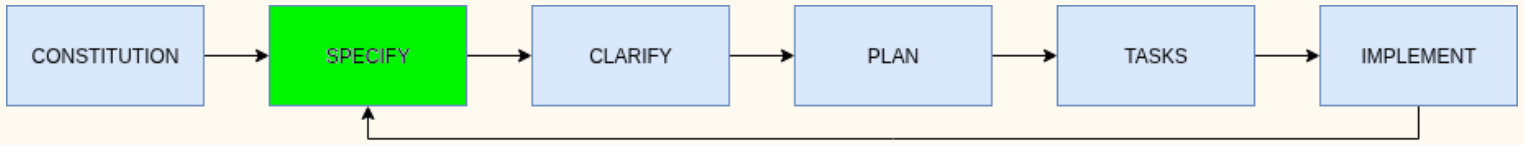


Principes attendus :

- Domain Driven Design
- Architecture Hexagonale
- API-First Contract
- TDD
- Java 21, Spring Boot 4

💻 Demo / VS Code

Rétro-Spécification



Exemple de prompt :

```
/speckit.specify En tant qu'architecte technique et fonctionnel
spécialisé en Java et Spring-Boot, effectue une analyse complète du code
de ce projet afin d'en faire une rétro spécification, appuies toi en
premier lieu sur les contrats OpenApi, AsyncApi et Les Modèles de
données. Effectue également un schéma d'architecture pour présenter les
interactions entre les différents composants. Rends-moi la main, afin
que je procède à sa validation avant de passer à l'étape suivante.
```



Spécification attendue :

- Entité clé: Client
- Endpoints principaux:
  - GET (liste)
  - POST (creation)
  - GET {id}
  - DELETE {id}
  - PUT {id}/adresse
  - PUT {id}/situation
- Validation adresse via IGN.
- PUT adresse -> Kafka
- Persistance: MongoDB

💻 Demo / VS Code

BDX I/O

D'autres fonctionnalités ?

Laissez libre court à votre imagination

BDX I/O

Exemples Backend

- Ajouter un numéro de téléphone
- Ajouter un email
- Mettre en place le contrôle JWT
- ...

Exemples Frontend

- Mettre en place les autres écrans
- Mettre en place un Logo
- Mettre en place l'authentification
- ...

BDX I/O

Conclusion
À retenir

Diverses Approches

BDX I/O

Diverses approches et divers outils permettent de mêtre en place du développement à partir de spécifications, et le choix dépend plus du niveau d'ambition souhaité que des technologies.

Critère	Mode Plan	OpenSpec	Spec-Kit	BMad
Prototype	⭐⭐⭐⭐⭐	⭐⭐⭐⭐☆	⭐⭐⭐☆☆	⭐⭐☆☆☆
Produit stratégique	⭐☆☆☆☆	⭐⭐☆☆☆	⭐⭐⭐⭐☆	⭐⭐⭐⭐⭐
Startup	⭐⭐⭐⭐⭐	⭐⭐⭐⭐☆	⭐⭐⭐☆☆	⭐⭐☆☆☆
Centre de services	⭐⭐☆☆☆	⭐⭐⭐☆☆	⭐⭐⭐⭐⭐	⭐⭐⭐☆☆
Produit long terme	⭐⭐☆☆☆	⭐⭐⭐☆☆	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
Coût IA relatif	⭐⭐⭐⭐⭐	⭐⭐⭐⭐☆	⭐⭐⭐☆☆	⭐☆☆☆☆

À retenir

BDX I/O

- **Spec-Kit**, un workflow simple et structuré pour le Specification-Driven Development
- **Essayez-le, testez-le, pratiquez-le** : c'est en l'utilisant qu'il prend tout son sens
- **Commencez petit** : le String Calculator Kata est un excellent point d'entrée
- **Attention aux limites** : les technos sorties après la date d'entraînement du LLM
- **Le rétro-engineering fonctionne très bien** sur des applications bien architecturées (Hexagonal, DDD, etc.)
- **Spécifiez en langage maternel** : vous serez souvent plus précis
- Pour mieux spécifier, **pensez design-first** : **OpenAPI, AsyncAPI, Data Models** avant le code
- TOUJOURS VÉRIFIER CE QUE GÉNÈRE VOTRE IA

sqli

BDX I/O



Thank You



Elevate. Digitally